

Ejercicio 1

Alberto Romero Orantes-Zurita

10 de marzo de 2026

1. Introducción

Para la realización del siguiente ejercicio, se utilizara Java como lenguaje de programación. Este ejercicio tiene como objetivo la creación y ejecución simultanea de dos partidas multijugador con el mismo número inicial (N) de jugadores. Dicho valor no se conocera hasta el comienzo de la partida.

2. Desarrollo

Para la realizacion del ejercicio se ha empleado la siguiente estructura:

2.1. FactoriaPartidayJugador.java

Este archivo contiene la interfaz publica FactoriaPartidayJugador en la cual se crea una partida y un jugador con un identificador asignado.

2.2. FactoriaCompetitiva.java

Este archivo contiene la clase FactoriaCompetitiva que implementa la clase FactoriaPartidayJugador mediante la cual crea una partida y un jugador competitivo. Incluye tambien las clases JugadorCompetitivo que hereda de Jugador al igual que PartidaCompetitiva que hereda de Partida.

2.3. FactoriaCasual.java

En este archivo encontramos la clase FactoriaCasual que implementa la clase FactoriaPartidayJugador en la que se crea una Partida y un jugador casual. Tambien encontramos la clase JugadorCasual que hereda de la clase Jugador al igual que la clase PartidaCasual que hereda de la clase Partida.

2.4. Jugador.java

En este archivo encontramos la clase abstracta Jugador la cual se compone de una varibale int 'id' de tipo 'protected'. Tambien encontramos el propio constructor de la clase y el metodo getId();

2.5. Partida.java

Este archivo contiene la clase abstracta Partida que usa Runnable para poder utilizar hebras para el inicio de las partidas de forma simultanea. Esta clase contiene un arraylist de tipo Jugador para almacenar los jugadores que componen cada partida. Tambien encontramos un constructor propio de la clase, un metodo agregarJugador() y por ultimo un metodo llamado run() encargado de simular la ejecucion de la partida.

2.6. Main.java

Por ultimo, encontramos en el archivo Main.java un metodo main() encargado de lanzar la ejecucion de la simulacion. En primer lugar se le pide al usuario el numero de jugadores que desee por partida, a continuacion se procede con la construccion de los objetos Factoria y Partida, en tercer lugar se le agregan los jugadores a dichas partidas y por ultimo se crean las hebras correspondientes a cada partida. Una vez se inician las hebras mediante hilo(Competitivo o Casual).start(), se unen mediante hilo(Competitivo o Casual).join().

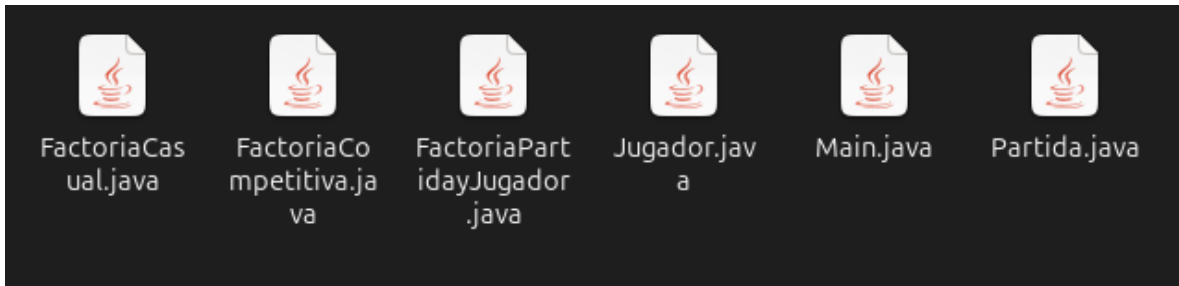


Figura 1: Archivos empleados.

3. Manejo de Concurrency (Hebras)

El enunciado exige que las partidas transcurran de forma simultánea, tengan una duración exacta de 60 segundos y que los abandonos se produzcan a la vez antes de terminar.

Para lograr esto, la clase abstracta **Partida** implementa la interfaz **Runnable**. El ciclo de vida programado en el método **run()** es el siguiente:

1. Iniciar la partida y pausar el hilo actual mediante `Thread.sleep(30000)` durante 30 segundos (simulando la primera mitad de la partida).
2. Ejecutar el método abstracto `simularAbandonosSimultaneos()`, el cual aplica el truncamiento matemático para eliminar de la lista al porcentaje de jugadores correspondiente de manera instantánea.
3. Pausar el hilo nuevamente durante 30 segundos para completar los 60 segundos totales exigidos.

4. Ejecucion del codigo

```
Introduce el número inicial de jugadores (N): 20

--- INICIANDO SIMULACIÓN DE 60 SEGUNDOS ---
[ PartidaCompetitiva ] Iniciada con: 20 jugadores
[ PartidaCasual ] Iniciada con: 20 jugadores
[Partida competitiva] 4 jugadores han abandonado la partida
[Partida casual] 2 jugadores han abandonado la partida
[ PartidaCompetitiva ] Finalizada. Jugadores restantes 16
[ PartidaCasual ] Finalizada. Jugadores restantes 18
--- AMBAS PARTIDAS HAN TERMINADO CORRECTAMENTE ---
```

Figura 2: Ejemplo de ejecucion con 20 jugadores.